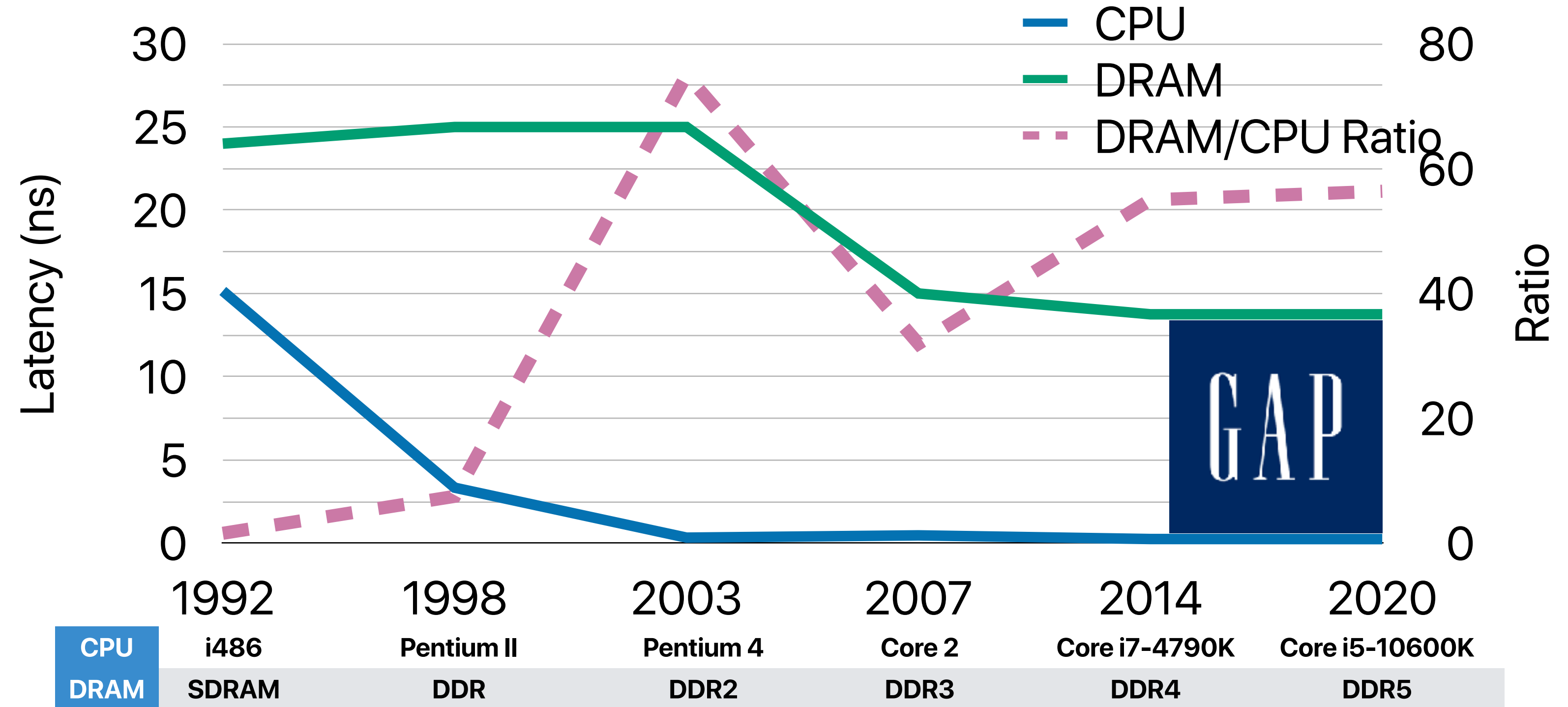


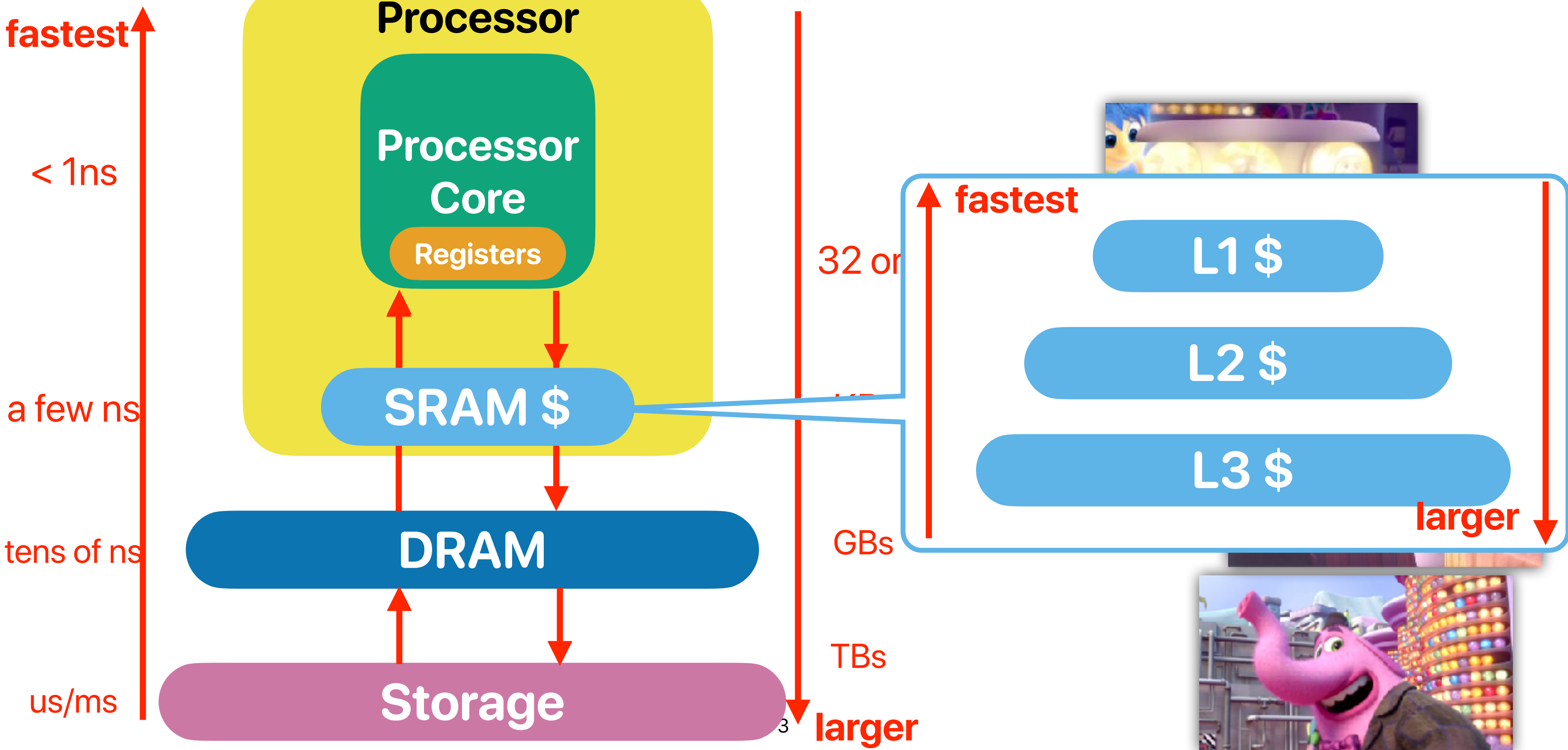
Memory Hierarchy (2): The A, B, Cs of Caches

Hung-Wei Tseng

Recap: the "latency" gap between CPU and DRAM



Recap: Memory Hierarchy



Recap: Data locality

- Which description about locality of arrays `matrix` and `vector` in the following code is the **most accurate**?

```
for(uint64_t i = 0; i < m; i++) {  
    result = 0;  
    for(uint64_t j = 0; j < n; j++) {  
        result += matrix[i][j]*vector[j];  
    }  
    output[i] = result;  
}
```

spatial locality:

`matrix[0][0]`, `matrix[0][1]`, `matrix[0][2]`, ...

`vector[0]`, `vector[1]`, ..., `vector[n]`

temporal locality:

reuse of `vector[0]`, `vector[1]`, ...,

- A. Access of `matrix` has temporal locality, `vector` has spatial locality
- B. Both `matrix` and `vector` have temporal locality, and `vector` also has spatial locality
- C. Access of `matrix` has spatial locality, `vector` has temporal locality
- D. Both `matrix` and `vector` have spatial locality and temporal locality
- E. Both `matrix` and `vector` have spatial locality, and `vector` also has temporal locality

Recap: Code also has locality

keep going to the
next instruction —
spatial locality

```
for(uint64_t i = 0; i < m; i++) {  
    result = 0;  
    for(uint64_t j = 0; j < n; j++) {  
        result += matrix[i][j]*vector[j];  
    }  
    output[i] = result;  
}
```

**repeat many times —
temporal locality!**

```
i = 0;  
while(i < m) {  
    result = 0;  
    j = 0;  
    while(j < n) {  
        a = matrix[i][j];  
        b = vector[j];  
        temp = a*b;  
        result = result + temp;  
    }  
    output[i] = result;  
    i++;  
}
```

Recap: inside out our memory hierarchy

- Memory access time is the most critical performance problem
 - One memory operation is as expensive as 50 arithmetic operations
 - Processor has to fetch instructions from memory
 - We have an average of 33% of data memory access instructions!
- Hierarchical caching with small amount of SRAMs will work if we can efficiently capture data and instructions
- Caching is possible! Most of time, we only work on a small amount of data!
 - Spatial locality
 - Temporal locality

Recap: Designing a hardware to exploit locality

- Spatial locality — application tends to visit nearby stuffs in the memory

We need to “cache consecutive memory locations” every time — the cache should store a “block” of code/data

- Temporal locality — application revisit the same thing again and again

We need to “cache frequently used memory blocks”

— the cache should store a few blocks several KBs

— the cache must be able to distinguish blocks • Data — program can read/write the same data many times (e.g., vectors in matrix-vector product)

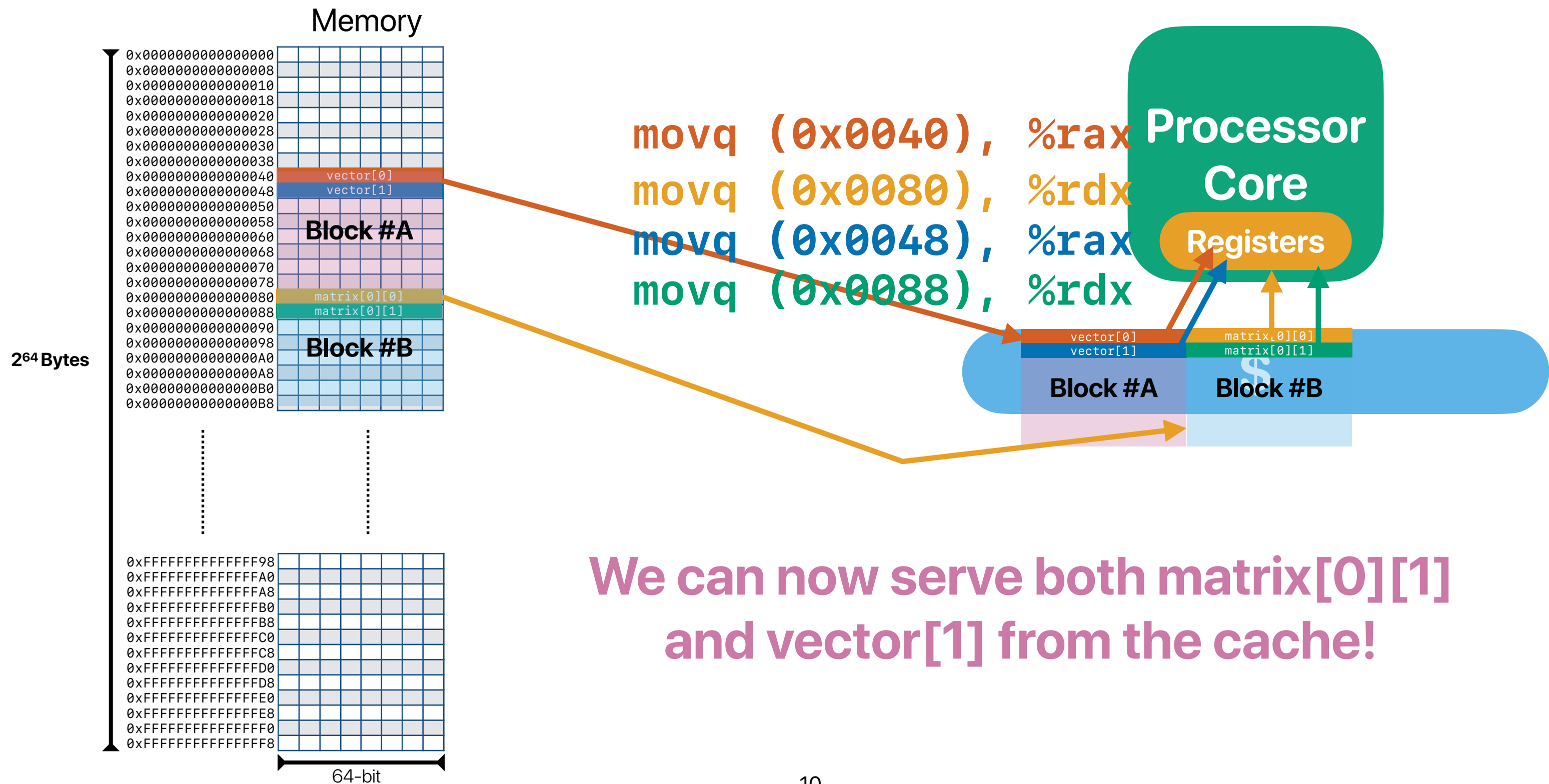
3:00

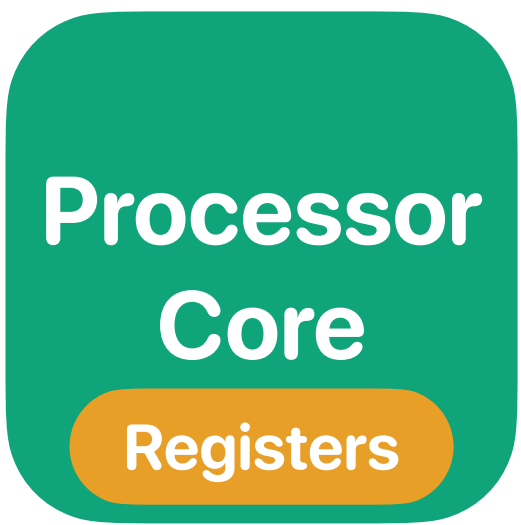
**How do you tell people where you
live in your home town?**

Outline

- Designing a cache that captures the predictability
- The A, B, Cs of caches
- Estimating how cache friendly is our code

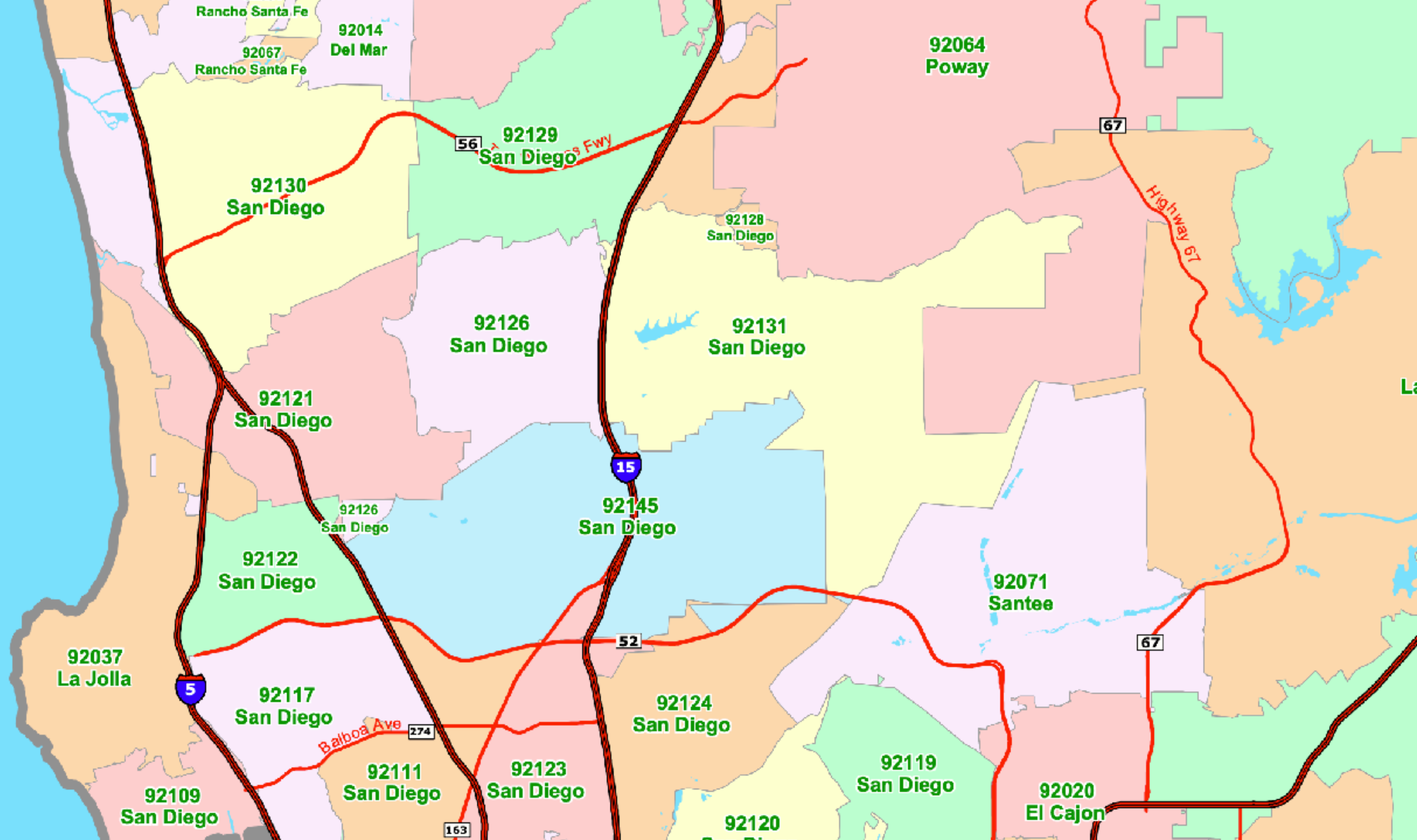
We need to cache multiple blocks of data!



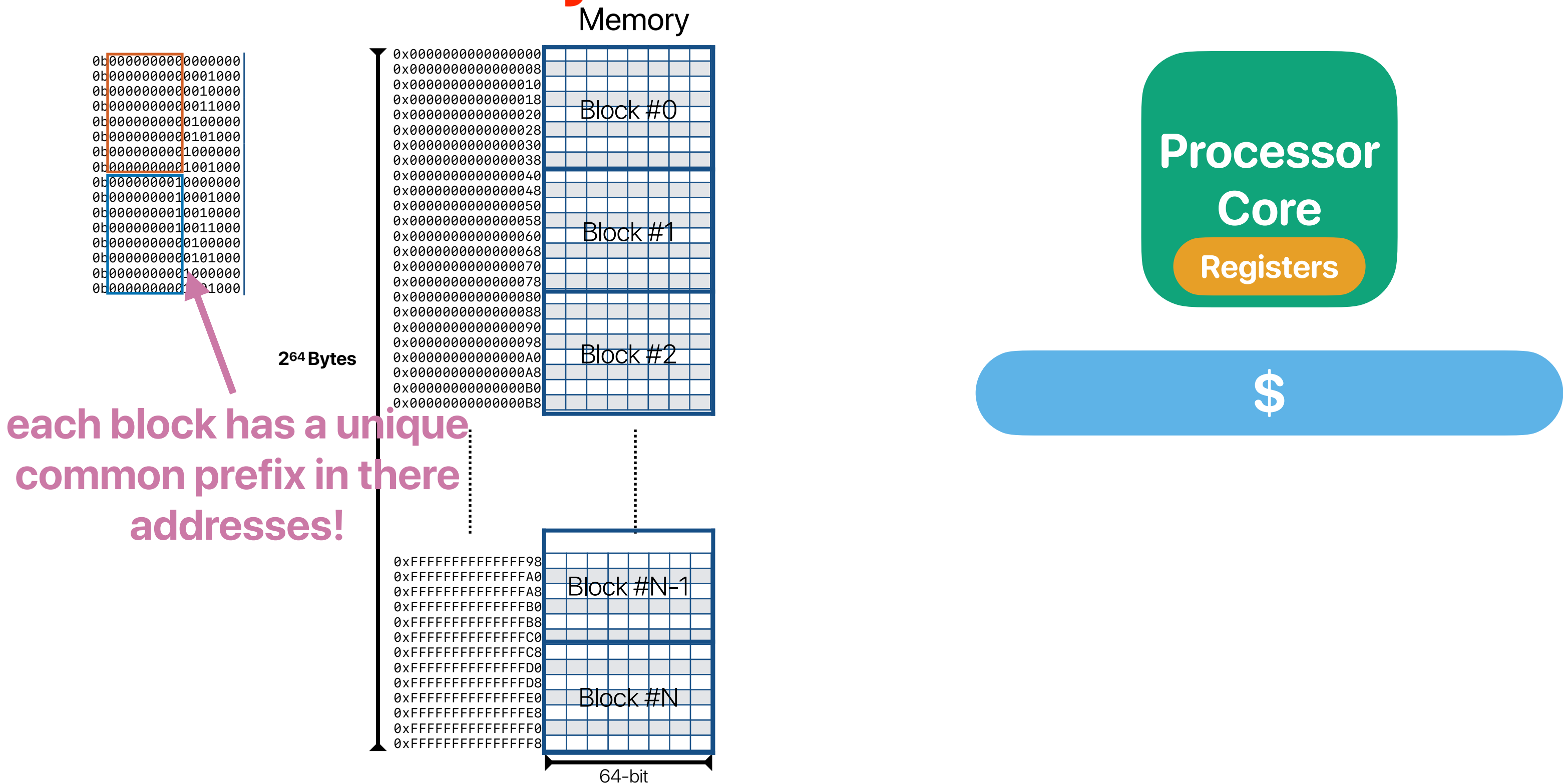


How to tell who is there?

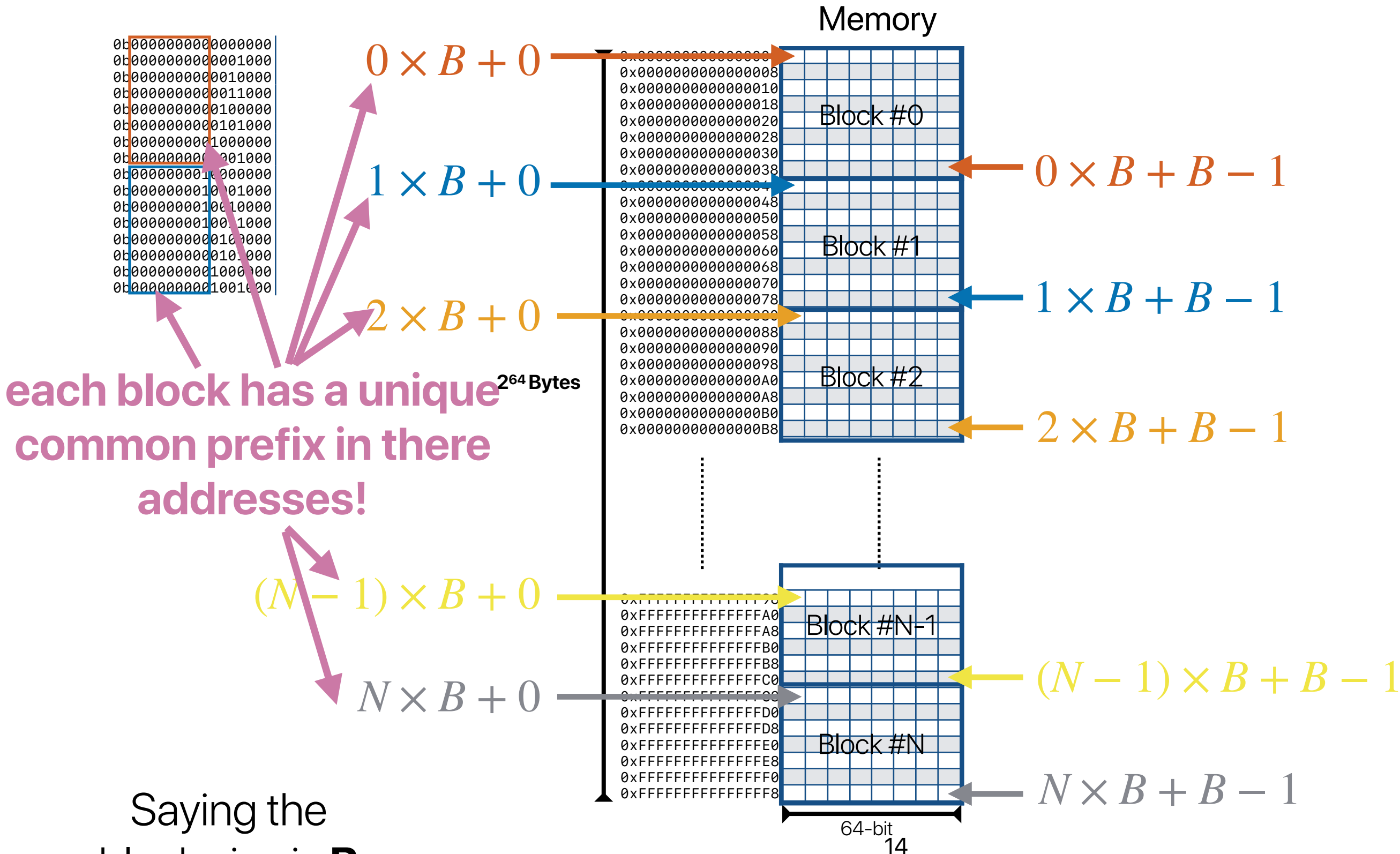
?															
?															
0123456789ABCDEF															
This is CSE142:															
Advanced Computer Architecture!															
This is CSE142:															
Advanced Computer Architecture!															
This is CSE142:															
Advanced Computer Architecture!															
This is CSE142:															
Advanced Computer Architecture!															
This is CSE142:															



Partition memory addresses into fix-sized chunks



Partition memory addresses into fix-sized chunks



Processor
Core

Registers

\$

Saying the
block size is **B**

Registers

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
0x0000	0x0001	0x0002	0x0003	0x0004	0x0005	0x0006	0x0007	0x0008	0x0009	0x000A	0x000B	0x000C	0x000D	0x000E	0x000F
01	2	3	4	5	6	7	8	9	A	B	C	D	E	F	



tag array



How to tell w

block offset

tag

1w 0x0008

1w 0x4048

0x404 not found,
go to lower-level memory

Tell if the block here can be used

Tell if the block here is modified

Valid Bit

Dirty Bit

tag

data

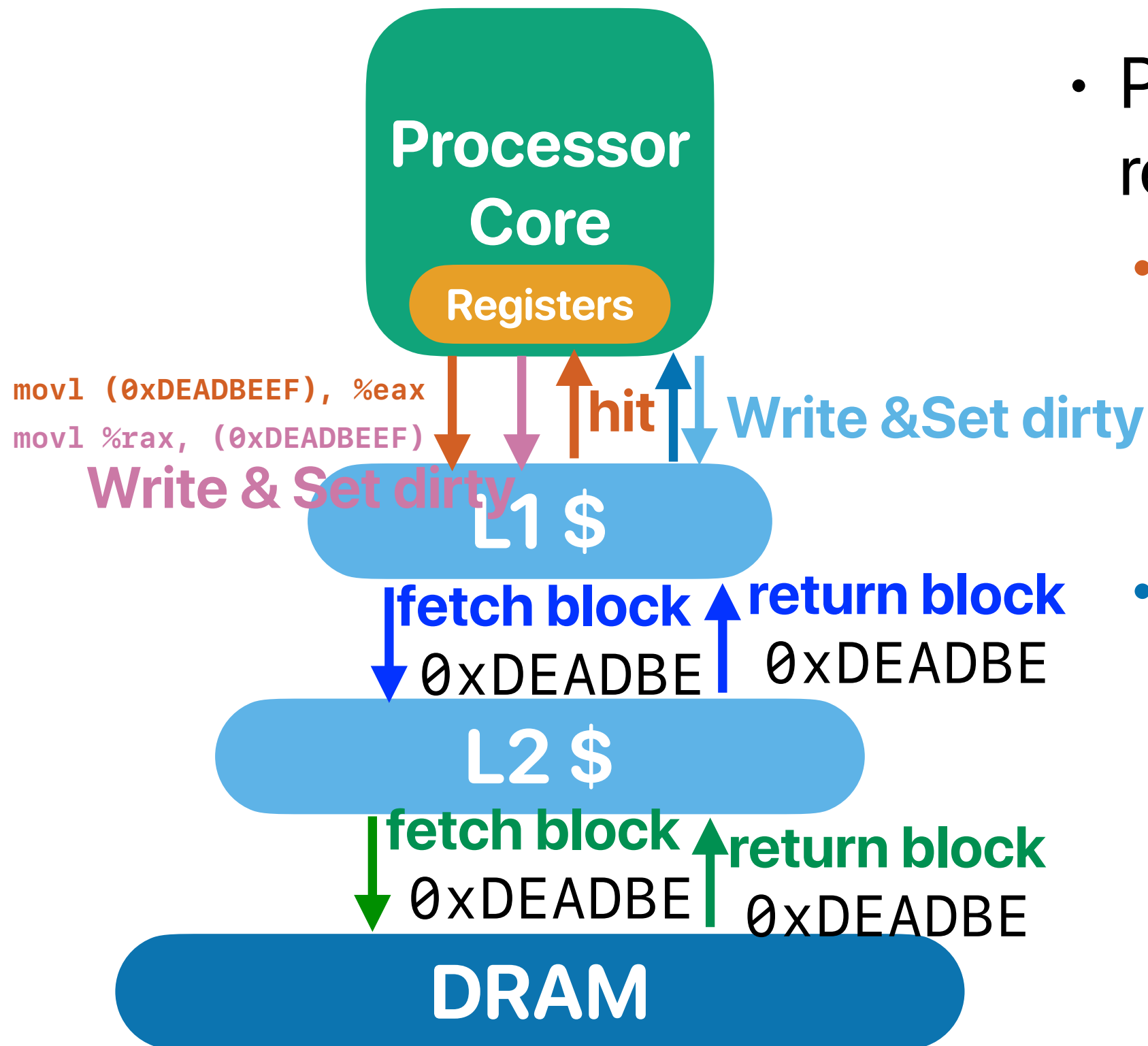
	1	1	0x000	This is CSE12:
	1	1	0x001	Advanced Compute
	1	0	0xF07	r Architecture!
	0	1	0x100	This is CSE142:
	1	1	0x310	Advanced Compute
	1	1	0x450	r Architecture!
	0	1	0x006	This is CSE142:
	0	1	0x537	Advanced Compute
	1	1	0x266	r Architecture!
ry	1	1	0x307	This is CSE142:
	0	1	0x265	Advanced Compute
	0	1	0x80A	r Architecture!
	1	1	0x620	This is CSE142:
	1	1	0x630	Advanced Compute
	1	0	0x705	r Architecture!
	0	1	0x216	This is CSE142:

Take-aways: designing caches

- Basic cache structures —
 - Caching in granularity of a block to capture spatial locality
 - Caching multiple blocks to keep frequently used data — temporal locality
 - Tags to distinguish cached blocks

**Put everything all together:
How cache interacts with CPU**

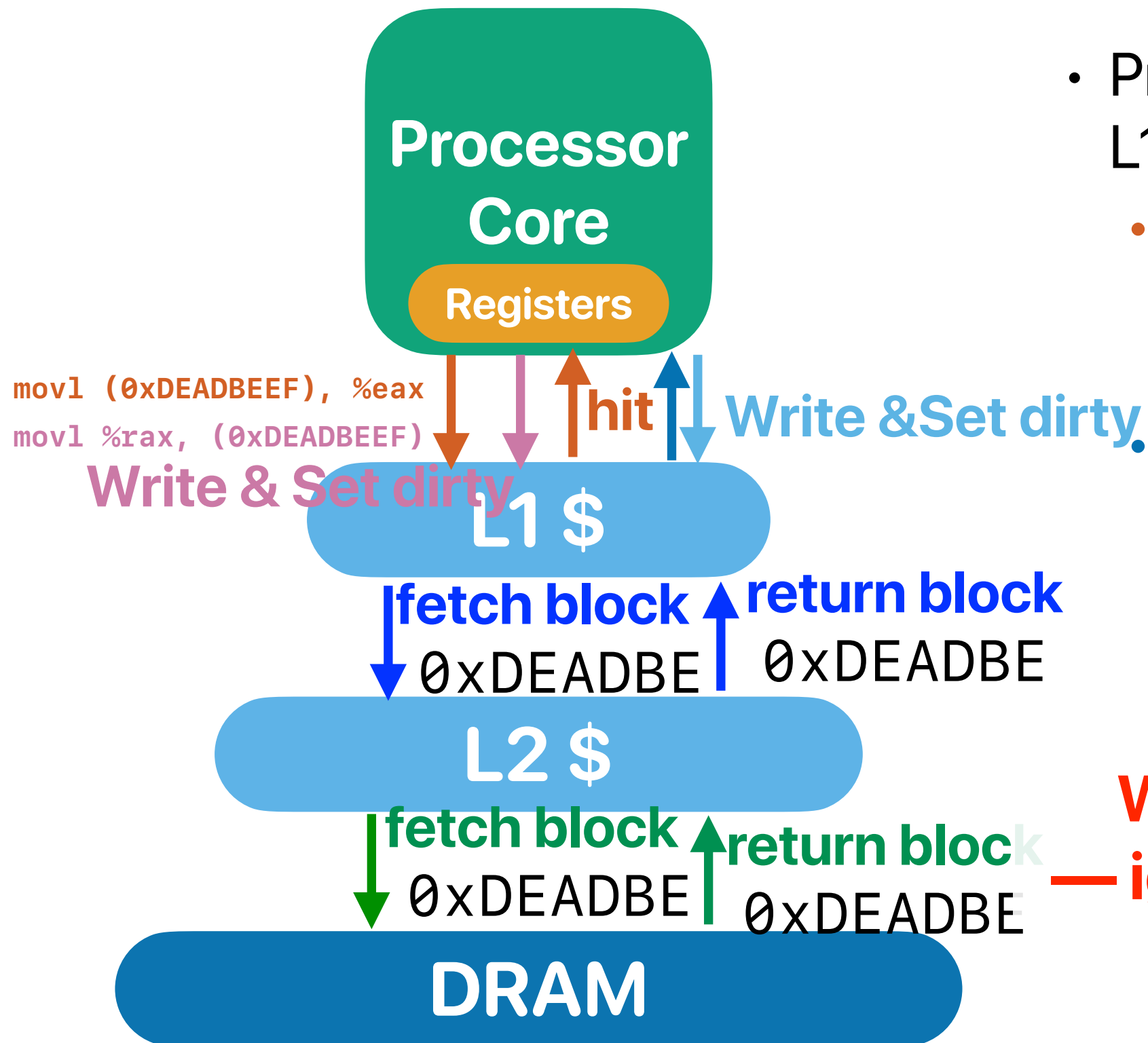
Processor/cache interaction



- Processor sends memory access request to L1-\$
 - **if hit & it's a read**
 - **Read:** return data
 - **Write:** Update "ONLY" in L1 and set DIRTY **Why don't we write to L2? — Too slow**
 - **if miss**
 - Fetch the requesting block from lower-level memory hierarchy and place in the cache
 - **Present the write "ONLY" in L1 and set DIRTY**

What if we run out of \$ blocks?

Processor/cache interaction

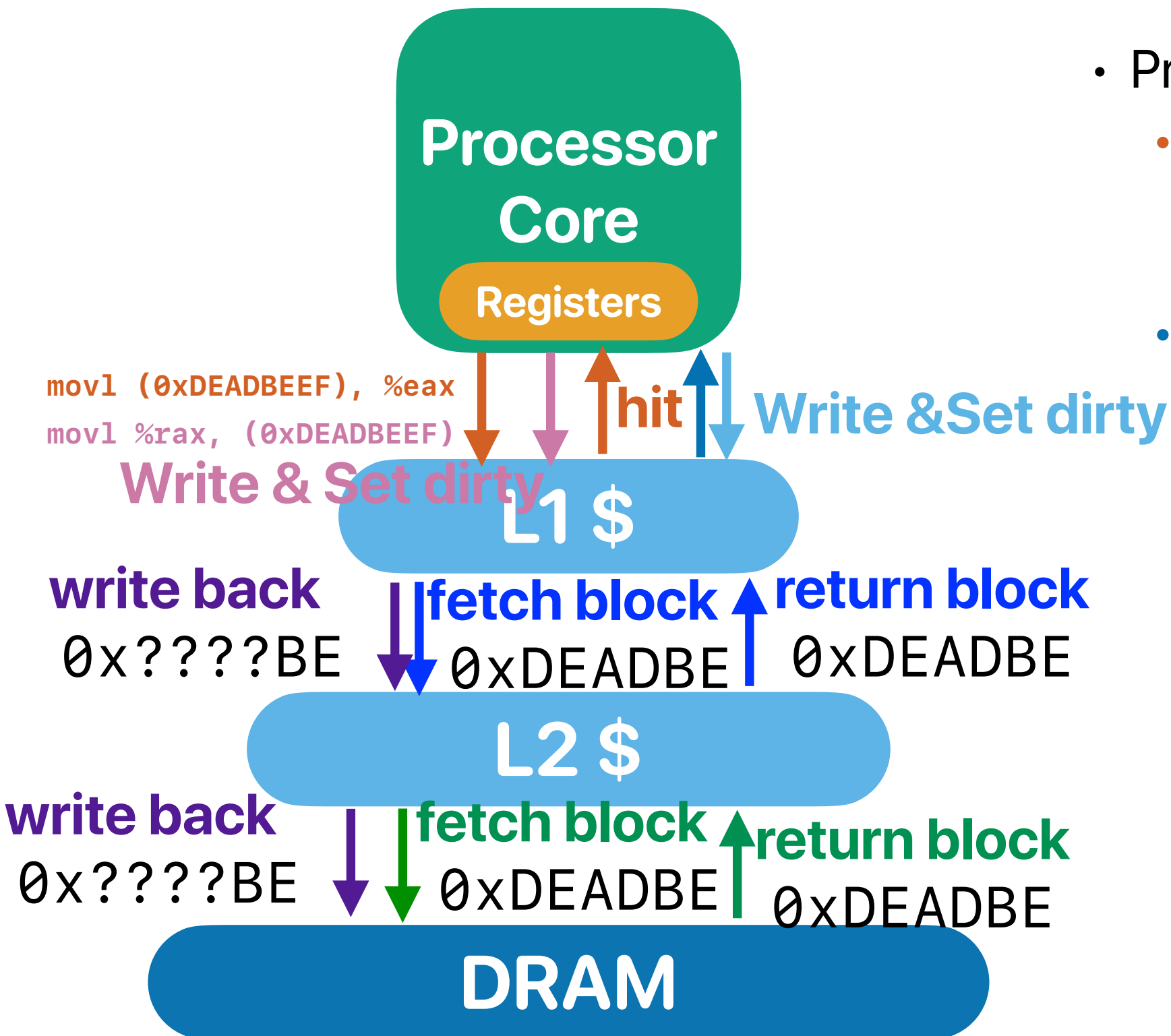


- Processor sends memory access request to L1-\$
 - **if hit & it's a read**
 - Read: return data
 - Write: Update "ONLY" in L1 and set DIRTY
 - **if miss**
 - If there an empty block — place the data there
 - If NOT (most frequent case) — select a **victim block**
 - Least Recently Used (LRU) policy

What if the victim block is modified?
— ignoring the update is not acceptable!

- Present the write "ONLY" in L1 and set DIRTY

Processor/cache interaction



- Processor sends memory access request to L1-\$
 - **if hit & it's a read**
 - **Read: return data**
 - **Write: Update "ONLY" in L1 and set DIRTY**
 - **if miss**
 - If there an empty block — place the data there
 - If NOT (most frequent case) — select a **victim block**
 - Least Recently Used (LRU) policy
 - If the victim block is "dirty" & "valid"
 - **Write back** the block to lower-level memory hierarchy
 - If write-back or fetching causes any miss, repeat the same process
 - Fetch the requesting block from lower-level memory hierarchy and place in the cache
 - **Present the write "ONLY" in L1 and set DIRTY**

Take-aways: designing caches

- Basic cache structures —
 - Caching in granularity of a block to capture spatial locality
 - Caching multiple blocks to keep frequently used data — temporal locality
 - Tags to distinguish cached blocks
- Hierarchical caching — data must be presented on the top level (L1) before the processor can use

How to tell w



block offset

tag

1w 0x0008

1w 0x4048

0x404 not found,
go to lower-level memory

The complexity of search the matching tag—
 $O(n)$ —will be slow if our cache size grows!

Can we search things faster?
—hash table! $O(1)$

Tell if the block here can be used

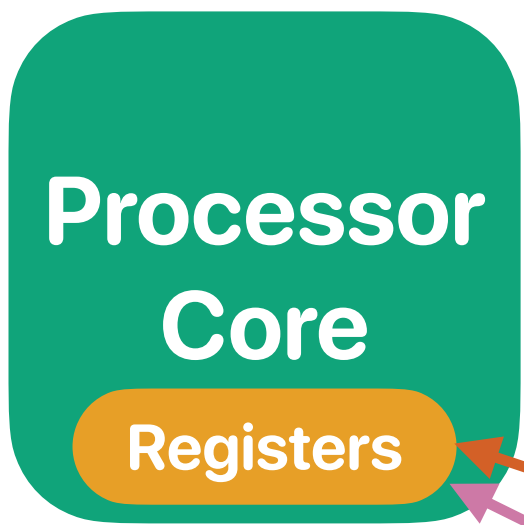
Tell if the block here is modified

Valid Bit
Dirty Bit

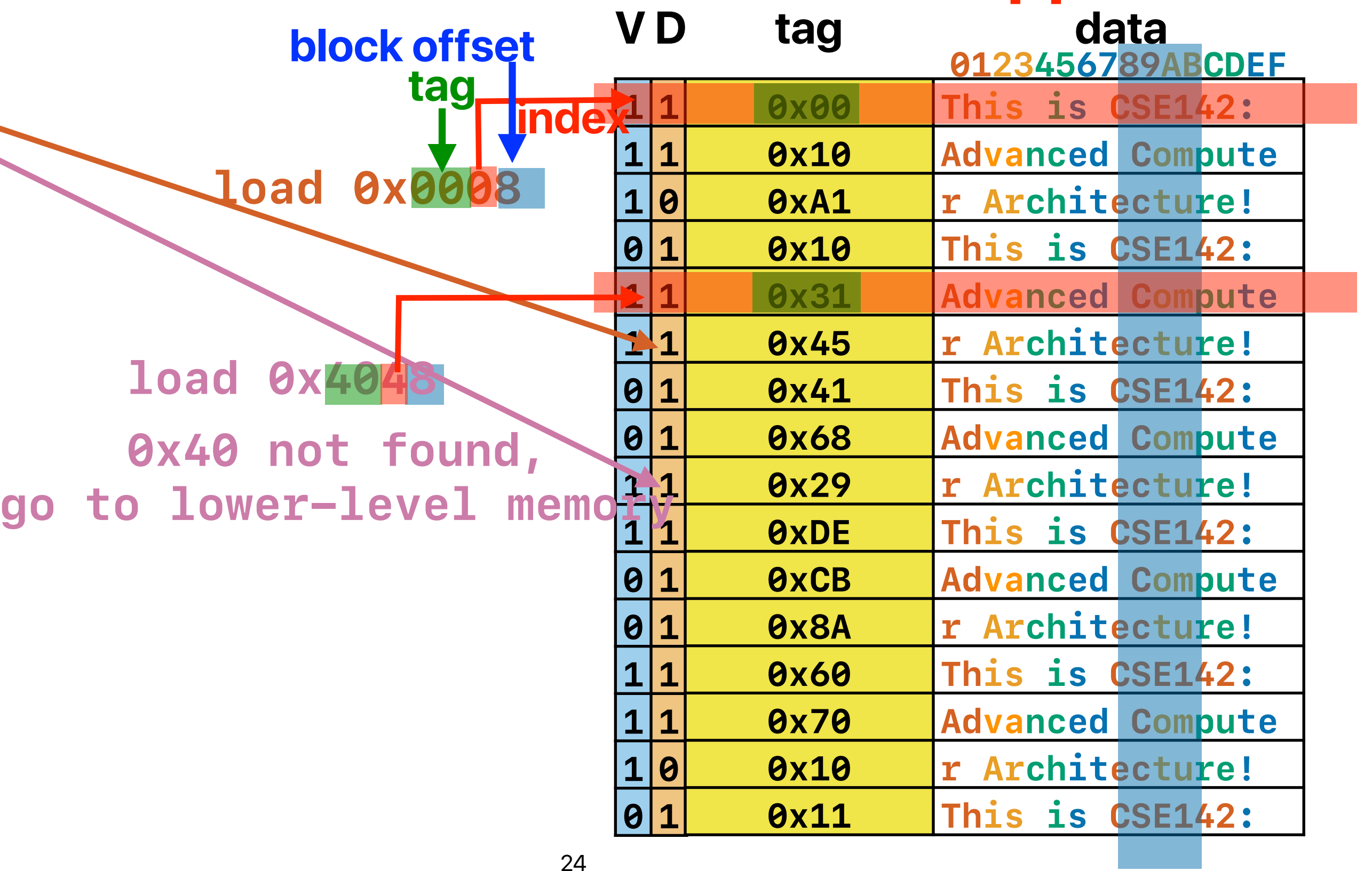
tag

data

				0123456789ABCDEF
1	1	0x000	This is CSE12:	
1	1	0x001	Advanced Compute	
1	0	0xF07	r Architecture!	
0	1	0x100	This is CSE142:	
1	1	0x310	Advanced Compute	
1	1	0x450	r Architecture!	
0	1	0x006	This is CSE142:	
0	1	0x537	Advanced Compute	
1	1	0x266	r Architecture!	
1	1	0x307	This is CSE142:	
0	1	0x265	Advanced Compute	
0	1	0x80A	r Architecture!	
1	1	0x620	This is CSE142:	
1	1	0x630	Advanced Compute	
1	0	0x705	r Architecture!	
0	1	0x216	This is CSE142:	

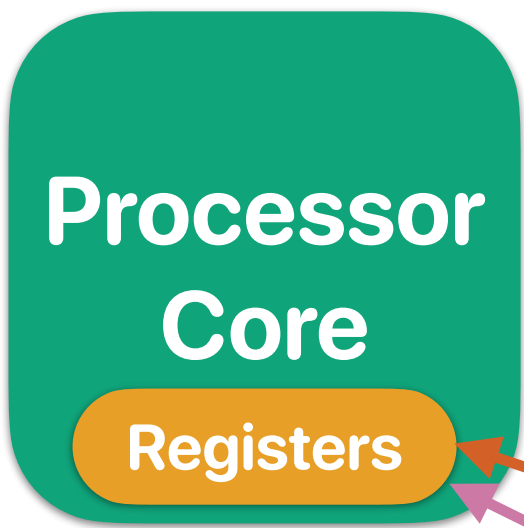


Hash-like structure — direct-mapped cache

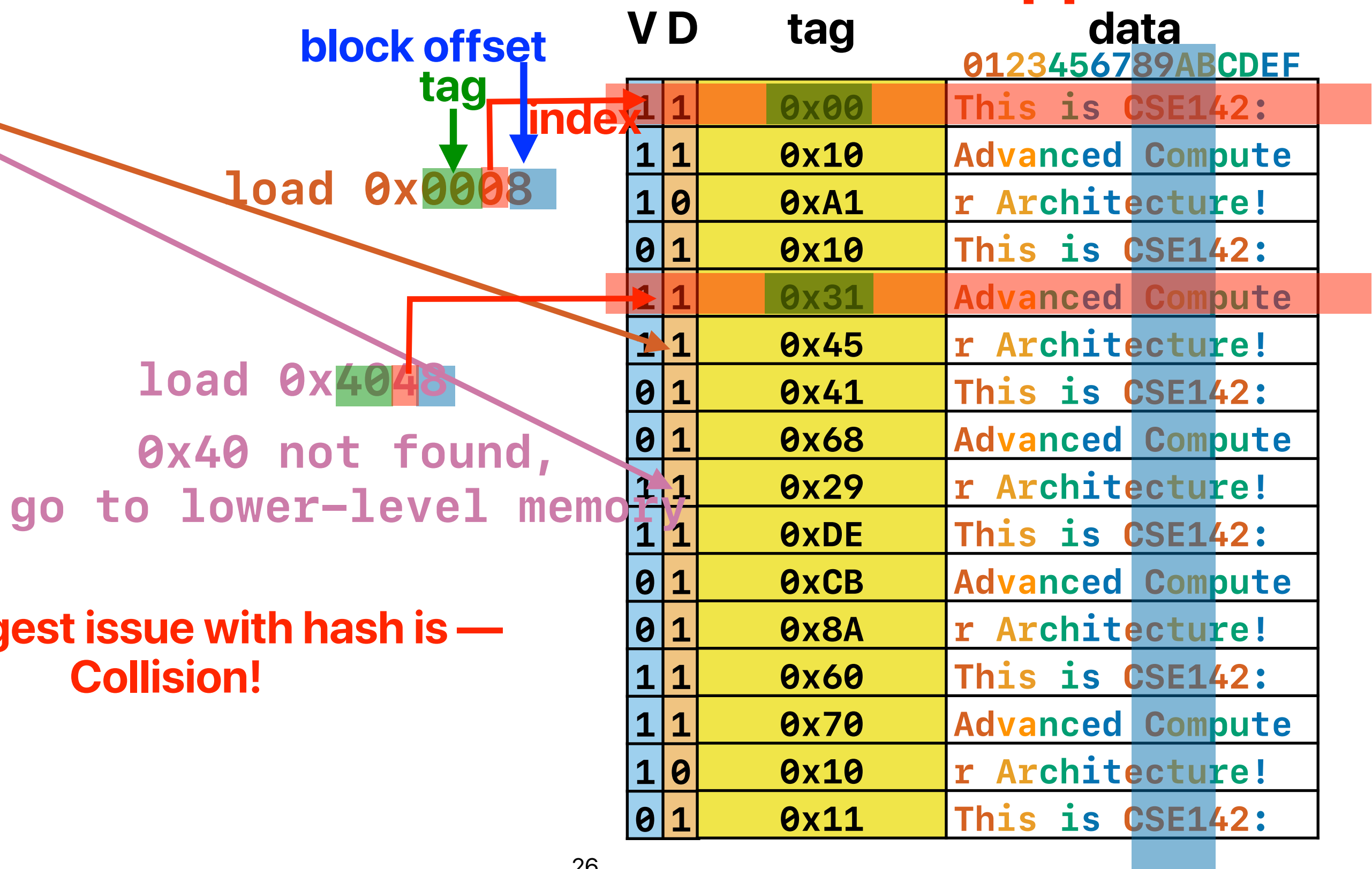


Take-aways: designing caches

- Basic cache structures —
 - Caching in granularity of a block to capture spatial locality
 - Caching multiple blocks to keep frequently used data — temporal locality
 - Tags to distinguish cached blocks
- Hierarchical caching — data must be presented on the top level (L1) before the processor can use
- Optimizing cache structures
 - Hash block into "sets" to reduce the search time



Hash-like structure — direct-mapped cache



The biggest issue with hash is — Collision!

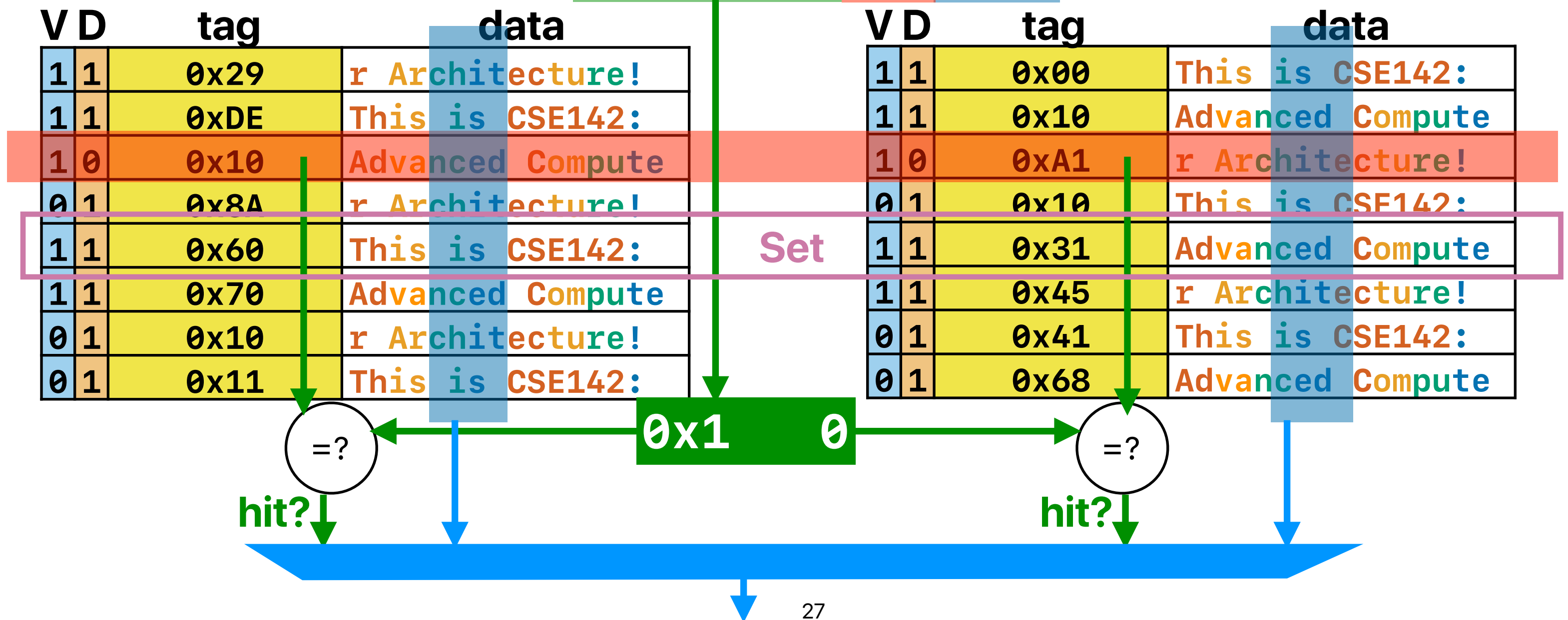
Way-associative cache

memory address: $0x0$ 8 2 4

set block

tag index offset

memory address: $0b00001000000100100$



Blocksize == Linesize

```
/sys/devices/system/cpu/cpu0/cache/index0/physical_line_partition:1
/sys/devices/system/cpu/cpu0/cache/index0/number_of_sets:64
/sys/devices/system/cpu/cpu0/cache/index0/ways_of_associativity:12
/sys/devices/system/cpu/cpu0/cache/index0/id:0
/sys/devices/system/cpu/cpu0/cache/index0/shared_cpu_list:0-1
/sys/devices/system/cpu/cpu0/cache/index0/type:Data
/sys/devices/system/cpu/cpu0/cache/index0/size:48K
/sys/devices/system/cpu/cpu0/cache/index0/level:1
/sys/devices/system/cpu/cpu0/cache/index0/coherency_line_size:64
/sys/devices/system/cpu/cpu0/cache/index0/shared_cpu_map:000003
```

Blocksize == Linesize

```
/sys/devices/system/cpu/cpu0/cache/index0/physical_line_partition:1
/sys/devices/system/cpu/cpu0/cache/index0/number of sets:64
/sys/devices/system/cpu/cpu0/cache/index0/ways of associativity:12
/sys/devices/system/cpu/cpu0/cache/index0/id:0
/sys/devices/system/cpu/cpu0/cache/index0/shared_cpu_list:0-1
/sys/devices/system/cpu/cpu0/cache/index0/type:Data
/sys/devices/system/cpu/cpu0/cache/index0/size:48K
/sys/devices/system/cpu/cpu0/cache/index0/level:1
/sys/devices/system/cpu/cpu0/cache/index0/coherency_line_size:64
/sys/devices/system/cpu/cpu0/cache/index0/shared_cpu_map:000003
```

Take-aways: designing caches

- Basic cache structures —
 - Caching in granularity of a block to capture spatial locality
 - Caching multiple blocks to keep frequently used data — temporal locality
 - Tags to distinguish cached blocks
- Hierarchical caching — data must be presented on the top level (L1) before the processor can use
- Optimizing cache structures
 - Hash block into "sets" to reduce the search time
 - Set-associativity to reduce the "collision" problem

Way-associative cache

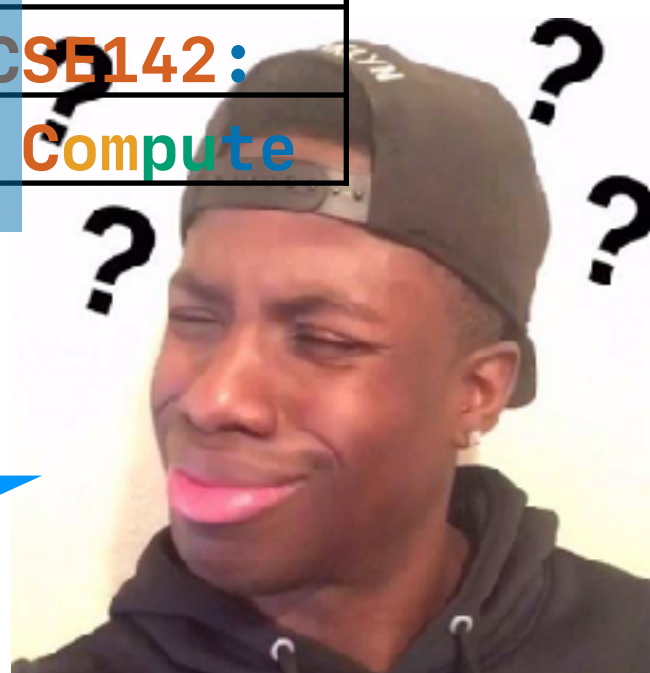
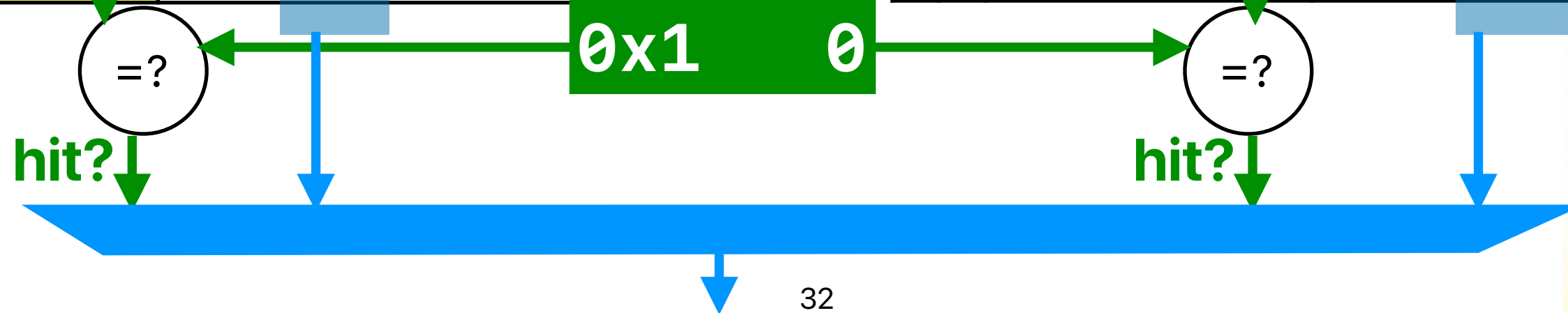
memory address: $0x0$ 8 2 4

set block

tag index offset

memory address: $0b00001000000100100$

V	D	tag	data		V	D	tag	data
1	1	$0x29$	r Architecture!		1	1	$0x00$	This is CSE142:
1	1	$0xDE$	This is CSE142:		1	1	$0x10$	Advanced Compute
1	0	$0x10$	Advanced Compute		1	0	$0xA1$	r Architecture!
0	1	$0x8A$	r Architecture!		0	1	$0x10$	This is CSE142:
1	1	$0x60$	This is CSE142:	Set	1	1	$0x31$	Advanced Compute
1	1	$0x70$	Advanced Compute		1	1	$0x45$	r Architecture!
0	1	$0x10$	r Architecture!		0	1	$0x41$	This is CSE142:
0	1	$0x11$	This is CSE142:		0	1	$0x68$	Advanced Compute



The A, B, Cs of your cache

$$C = ABS$$

- **C: Capacity** in data arrays
- **A: Way-Associativity** — how many blocks within a set
 - N-way: N blocks in a set, $A = N$
 - 1 for direct-mapped cache
- **B: Block Size (Linesize)**
 - How many bytes in a block
- **S: Number of Sets:**
 - A set contains blocks sharing the same index
 - 1 for fully associate cache



Corollary of $C = ABS$

memory address: 0b 

- number of bits in **block** offset — $\lg(\mathbf{B})$
- number of bits in **set** index: $\lg(\mathbf{S})$
- tag bits: $\text{address_length} - \lg(\mathbf{S}) - \lg(\mathbf{B})$
 - address_length is N bits for N-bit machines (e.g., 64-bit for 64-bit machines)
- $(\text{address} / \text{block_size}) \% \mathbf{S} = \text{set index}$



NVIDIA Tegra Orin

- L1 data (D-L1) cache configuration of NVIDIA Tegra Orin
 - Size 64KB, 4-way set associativity, 64B block
 - Assume 64-bit memory address

Which of the following is correct?

- A. Tag is 49 bits
- B. Index is 8 bits
- C. Offset is 7 bits
- D. The cache has 1024 sets
- E. None of the above

A screenshot of a Pollev poll interface. It shows a list of five options, each with a corresponding input field for a response. The options are labeled A, B, C, D, and E. The input fields are currently empty.

NVIDIA Tegra Orin

- L1 data (D-L1) cache configuration of NVIDIA Tegra Orin
 - Size 64KB, 4-way set associativity, 64B block
 - Assume 64-bit memory address

Which of the following is correct?

- A. Tag is 49 bits
- B. Index is 8 bits**
- C. Offset is 7 bits
- D. The cache has 1024 sets
- E. None of the above

$$C = A \times B \times S$$

$$64 \times 1024 = 4 \times 64 \times S$$

$$S = 256$$

$$\text{Offset} = \log_2(64) = 6$$

$$\text{Index} = \log_2(256) = 8$$

$$\text{Tag} = 64 - 8 - 6 = 50$$



intel Core i7

- L1 data (D-L1) cache configuration of Core i7
 - Size 48KB, 12-way set associativity, 64B block
 - Assume 64-bit memory address
 - Which of the following is **NOT** correct?
 - A. Tag is 52 bits
 - B. Index is 6 bits
 - C. Offset is 6 bits
 - D. The cache has 128 sets
 - E. All of the above are correct

A screenshot of a Pollev poll interface. It shows a list of five input boxes, each preceded by a letter (A, B, C, D, E). The boxes are currently empty, indicating that no answers have been submitted yet.

intel Core i7

- L1 data (D-L1) cache configuration of Core i7
 - Size 48KB, 12-way set associativity, 64B block
 - Assume 64-bit memory address
 - Which of the following is **NOT** correct?
 - A. Tag is 52 bits
 - B. Index is 6 bits
 - C. Offset is 6 bits
 - D. The cache has 128 sets**
 - E. All of the above are correct

$$C = A \times B \times S$$

$$48 \times 1024 = 12 \times 64 \times S$$

$$S = 64$$

$$Offset = \log_2(64) = 6$$

$$Index = \log_2(12) = 3.58$$

$$Tag = 64 - 6 - 6 = 52$$

Take-aways: designing caches

- Basic cache structures —
 - Caching in granularity of a block to capture spatial locality
 - Caching multiple blocks to keep frequently used data — temporal locality
 - Tags to distinguish cached blocks
- Hierarchical caching — data must be presented on the top level (L1) before the processor can use
- Optimizing cache structures
 - Hash block into "sets" to reduce the search time
 - Set-associativity to reduce the "collision" problem
- $C = A B S$
 - C: capacity
 - A: Associativity
 - S: Number of sets
 - $\lg(S)$: Number of bits in set index
 - $\lg(B)$: Number of bits in block offset

Simulate the cache!

Simulate a direct-mapped cache

- A direct mapped (1-way) cache with 256 bytes total capacity, a block size of 16 bytes

- # of blocks = $\frac{256}{16} = 16$
- $\lg(16) = 4$: 4 bits are used for the index
- $\lg(16) = 4$: 4 bits are used for the byte offset
- The tag is $64 - (4 + 4) = 56$ bits

- For example: $0x \quad 8 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 8 \quad 0$
= $0b \quad 1000 \quad 0000 \quad 0000 \quad 0000 \quad 0000 \quad 0000 \quad 1000 \quad 0000$


Matrix vector revisited

```
for(uint64_t i = 0; i < m; i++) {  
    result = 0;  
    for(uint64_t j = 0; j < n; j++) {  
        result += matrix[i][j]*vector[j];  
    }  
    output[i] = result;  
}
```

Matrix vector revisited

```
for(uint64_t i = 0; i < m; i++) {
    result = 0;
    for(uint64_t j = 0; j < n; j++) {
        result += matrix[i][j]*vector[j];
    }
    output[i] = result;
}
```

tagindex

	Address (Hex)	Address (Binary)
&a[0][0]	0x558FE0A1D330	0b10101011000111111100000101000011101001100110000
&b[0]	0x558FE0A1DC30	0b10101011000111111100000101000011101110000110000
&a[0][1]	0x558FE0A1D338	0b10101011000111111100000101000011101001100111000
&b[1]	0x558FE0A1DC38	0b101010110001111111000001010000111011100000111000
&a[0][2]	0x558FE0A1D340	0b10101011000111111100000101000011101001101000000
&b[2]	0x558FE0A1DC40	0b101010110001111111000001010000111011100001000000
&a[0][3]	0x558FE0A1D348	0b10101011000111111100000101000011101001101001000
&b[3]	0x558FE0A1DC48	0b101010110001111111000001010000111011100001001000
&a[0][4]	0x558FE0A1D350	0b10101011000111111100000101000011101001101010000
&b[4]	0x558FE0A1DC50	0b101010110001111111000001010000111011100001010000
&a[0][5]	0x558FE0A1D358	0b10101011000111111100000101000011101001101011000
&b[5]	0x558FE0A1DC58	0b101010110001111111000001010000111011100001011000
&a[0][6]	0x558FE0A1D360	0b10101011000111111100000101000011101001101100000
&b[6]	0x558FE0A1DC60	0b101010110001111111000001010000111011100001100000
&a[0][7]	0x558FE0A1D368	0b10101011000111111100000101000011101001101101000
&b[7]	0x558FE0A1DC68	0b101010110001111111000001010000111011100001101000
&a[0][8]	0x558FE0A1D370	0b10101011000111111100000101000011101001101110000
&b[8]	0x558FE0A1DC70	0b1010101100011111110000010100001110111000001110000
&a[0][9]	0x558FE0A1D378	0b10101011000111111100000101000011101001101111000
&b[9]	0x558FE0A1DC78	0b1010101100011111110000010100001110111000001111000

Simulate a direct-mapped cache

tag index

	V	D	Tag	Data
0	0	0		
1	0	0		
2	0	0		
3	1	0	0x558FE0A1DC	b[0], b[1]
4	1	0	0x558FE0A1DC	b[2], b[3]
5	0	0		
6	0	0		
7	0	0		
8	0	0		
9	0	0		
10	0	0		
11	0	0		
12	0	0		
13	0	0		
14	0	0		
15	0	0		

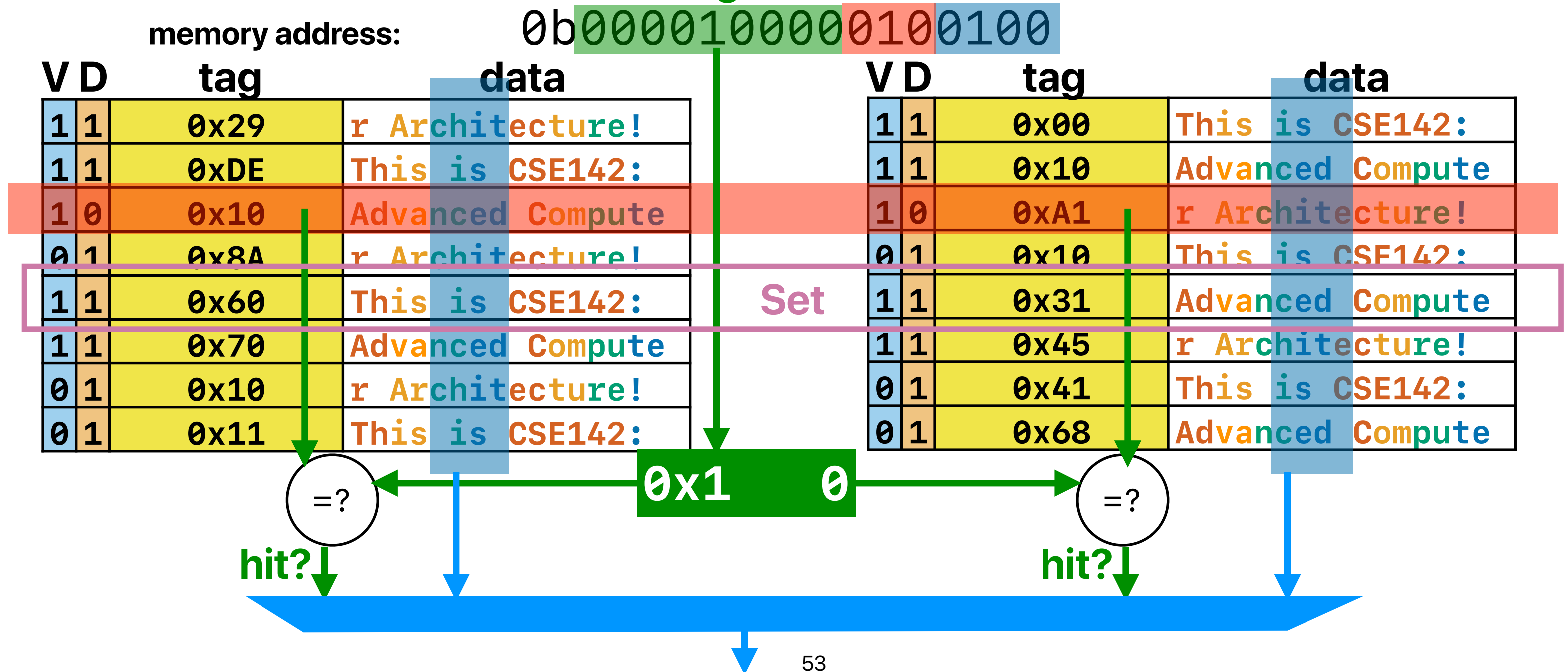
This cache doesn't work!!!
— collisions!

Address (Hex)		
&a[0][0]	0x558FE0A1D330	miss
&b[0]	0x558FE0A1DC30	miss
&a[0][1]	0x558FE0A1D338	miss
&b[1]	0x558FE0A1DC38	miss
&a[0][2]	0x558FE0A1D340	miss
&b[2]	0x558FE0A1DC40	miss
&a[0][3]	0x558FE0A1D348	miss
&b[3]	0x558FE0A1DC48	miss
&a[0][4]	0x558FE0A1D350	miss
&b[4]	0x558FE0A1DC50	miss
&a[0][5]	0x558FE0A1D358	miss
&b[5]	0x558FE0A1DC58	miss
&a[0][6]	0x558FE0A1D360	miss
&b[6]	0x558FE0A1DC60	miss
&a[0][7]	0x558FE0A1D368	miss
&b[7]	0x558FE0A1DC68	miss
&a[0][8]	0x558FE0A1D370	miss
&b[8]	0x558FE0A1DC70	miss
&a[0][9]	0x558FE0A1D378	
&b[9]	0x558FE0A1DC78	

Way-associative cache

memory address: 0x0 8 2 4

tag set index block offset



Now, 2-way, same-sized cache

- A 2-way cache with 256 bytes total capacity, a block size of 16 bytes

- # of blocks = $\frac{256}{16} = 16$
- # of sets = $\frac{16}{2} = 8$ (2-way: 2 blocks in a set)
- $\lg(8) = 3$: 3 bits are used for the index
- $\lg(16) = 4$: 4 bits are used for the byte offset
- The tag is $64 - (4 + 4) = 56$ bits
- For example: $0x$ 8 0 0 0 0 0 8 0

= 0b1000 0000 0000 0000 0000 0000 0000 1000 0000

The diagram shows a 64-bit address split into three fields: a 56-bit tag (red), a 3-bit index (blue), and a 4-bit offset (black). The tag field is the first 56 bits, the index field is the next 3 bits, and the offset field is the last 4 bits. The address is 0b1000 0000 0000 0000 0000 0000 0000 1000 0000. The tag is 1000 0000 0000 0000 0000 0000 0000, the index is 100, and the offset is 0000.

tag index offset

Matrix vector revisited

		tag	index		tag	index
		Address (Hex)		Address (Binary)		
<pre>for(uint64_t i = 0; i < m; i++) { result = 0; for(uint64_t j = 0; j < n; j++) { result += matrix[i][j]*vector[j]; } output[i] = result; }</pre>	&a[0][0]	0x558FE0A1D330	0b	10101011000111111100000101000011101001100110000		
	&b[0]	0x558FE0A1DC30	0b	10101011000111111100000101000011101110000110000		
	&a[0][1]	0x558FE0A1D338	0b	10101011000111111100000101000011101001100111000		
	&b[1]	0x558FE0A1DC38	0b	10101011000111111100000101000011101110000111000		
	&a[0][2]	0x558FE0A1D340	0b	10101011000111111100000101000011101001101000000		
	&b[2]	0x558FE0A1DC40	0b	10101011000111111100000101000011101110001000000		
	&a[0][3]	0x558FE0A1D348	0b	10101011000111111100000101000011101001101001000		
	&b[3]	0x558FE0A1DC48	0b	10101011000111111100000101000011101110001001000		
	&a[0][4]	0x558FE0A1D350	0b	10101011000111111100000101000011101001101010000		
	&b[4]	0x558FE0A1DC50	0b	10101011000111111100000101000011101110001010000		
	&a[0][5]	0x558FE0A1D358	0b	10101011000111111100000101000011101001101011000		
	&b[5]	0x558FE0A1DC58	0b	10101011000111111100000101000011101110001011000		
	&a[0][6]	0x558FE0A1D360	0b	10101011000111111100000101000011101001101100000		
	&b[6]	0x558FE0A1DC60	0b	10101011000111111100000101000011101110001100000		
	&a[0][7]	0x558FE0A1D368	0b	10101011000111111100000101000011101001101101000		
	&b[7]	0x558FE0A1DC68	0b	10101011000111111100000101000011101110001101000		
	&a[0][8]	0x558FE0A1D370	0b	10101011000111111100000101000011101001101110000		
	&b[8]	0x558FE0A1DC70	0b	10101011000111111100000101000011101110001110000		
	&a[0][9]	0x558FE0A1D378	0b	10101011000111111100000101000011101001101111000		
	&b[9]	0x558FE0A1DC78	0b	10101011000111111100000101000011101110001111000		

Simulate a 2-way cache

V	D	Tag	Data	V	D	Tag	Data
0	0			0	0		
0	0			0	0		
0	0			0	0		
1	0	0xAB1FC143A6	a[0][0], a[0][1]	1	0	0xAB1FC143B8	b[0], b[1]
1	0	0xAB1FC143A6	a[0][2], a[0][3]	1	0	0xAB1FC143B8	b[2], b[3]
0	0			0	0		
0	0			0	0		
0	0			0	0		

	Address (Hex)	Tag	Index	
&a[0][0]	0x558FE0A1D330	0xAB1FC143A6	0x3	miss
&b[0]	0x558FE0A1DC30	0xAB1FC143B8	0x3	miss
&a[0][1]	0x558FE0A1D338	0xAB1FC143A6	0x3	hit
&b[1]	0x558FE0A1DC38	0xAB1FC143B8	0x3	hit
&a[0][2]	0x558FE0A1D340	0xAB1FC143A6	0x4	miss
&b[2]	0x558FE0A1DC40	0xAB1FC143B8	0x4	miss
&a[0][3]	0x558FE0A1D348	0xAB1FC143A6	0x4	hit
&b[3]	0x558FE0A1DC48	0xAB1FC143B8	0x4	hit
&a[0][4]	0x558FE0A1D350	0xAB1FC143A6	0x5	miss
&b[4]	0x558FE0A1DC50	0xAB1FC143B8	0x5	miss
&a[0][5]	0x558FE0A1D358	0xAB1FC143A6	0x5	hit
&b[5]	0x558FE0A1DC58	0xAB1FC143B8	0x5	hit
&a[0][6]	0x558FE0A1D360	0xAB1FC143A6	0x6	miss
&b[6]	0x558FE0A1DC60	0xAB1FC143B8	0x6	miss
&a[0][7]	0x558FE0A1D368	0xAB1FC143A6	0x6	hit
&b[7]	0x558FE0A1DC68	0xAB1FC143B8	0x6	hit
&a[0][8]	0x558FE0A1D370	0xAB1FC143A6	0x7	miss
&b[8]	0x558FE0A1DC70	0xAB1FC143B8	0x7	miss
&a[0][9]	0x558FE0A1D378	0xAB1FC143A6	0x7	hit
&b[9]	0x558FE0A1DC78	0xAB1FC143B8	0x7	hit



NVIDIA Tegra Orin

- D-L1 Cache configuration of NVIDIA Tegra Orin
 - Size 64KB, 4-way set associativity, 64B block, LRU policy, write-allocate, write-back, and assuming 64-bit address.

```
double a[16384], b[16384], c[16384], d[16384], e[16384];
/* a = 0x10000, b = 0x20000, c = 0x30000, d = 0x40000, e = 0x50000 */
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
    //load a[i], b[i], c[i], d[i] and then store to e[i]
}
```

What's the data cache miss rate for this code?

- A. 12.5%
- B. 56.25%
- C. 66.67%
- D. 68.75%
- E. 100%



NVIDIA Tegra Orin

100% miss rate!

- Size 64KB, 4-way set associativity, 64B block, LRU policy, write-allocate, write-back, and assuming 64-bit address.

```
double a[16384], b[16384], c[16384], d[16384], e[16384];
/* a = 0x10000, b = 0x20000, c = 0x30000, d = 0x40000, e = 0x50000 */
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
    //load a[i], b[i], c[i], d[i] and then store to e[i]
```

C = ABS

32KB = 4 * 64 * S

S = 256

offset = lg(64) = 6 bits

index = lg(256) = 8 bits

tag = the rest bits

	Address (Hex)	Address in binary			Tag	Index	Hit? Miss?	Replace?
a[0]	0x10000	0b000100	00000000	000000	0x4	0x0	Miss	
b[0]	0x20000	0b001000	00000000	000000	0x8	0x0	Miss	
c[0]	0x30000	0b001100	00000000	000000	0xC	0x0	Miss	
d[0]	0x40000	0b010000	00000000	000000	0x10	0x0	Miss	
e[0]	0x50000	0b010100	00000000	000000	0x14	0x0	Miss	a[0-7]
a[1]	0x10008	0b000100	00000000	001000	0x4	0x0	Miss	b[0-7]
b[1]	0x20008	0b001000	00000000	001000	0x8	0x0	Miss	c[0-7]
c[1]	0x30008	0b001100	00000000	001000	0xC	0x0	Miss	d[0-7]
d[1]	0x40008	0b010000	00000000	001000	0x10	0x0	Miss	e[0-7]
e[1]	0x50008	0b010100	00000000	001000	0x14	0x0	Miss	a[0-7]
⋮	⋮	⋮			⋮	⋮	⋮	⋮

NVIDIA Tegra X1

- D-L1 Cache configuration of NVIDIA Tegra X1
 - Size 32KB, 4-way set associativity, 64B block, LRU policy, write-allocate, write-back, and assuming 64-bit address.

```
double a[16384], b[16384], c[16384], d[16384], e[16384];
/* a = 0x10000, b = 0x20000, c = 0x30000, d = 0x40000, e = 0x50000 */
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
    //load a[i], b[i], c[i], d[i] and then store to e[i]
}
```

What's the data cache miss rate for this code?

- A. 12.5%
- B. 56.25%
- C. 66.67%
- D. 68.75%
- E. 100%

Announcement

- Reading quiz #4 due **tomorrow** before the lecture — all reading quizzes are on gradescope
- Check your grades on https://www.escalab.org/my_grades
- Assignment #2 due **this Sunday**
 - You should run the performance measurement yourself and calculate results based on that — everyone should have a different answer
 - All questions this time require **correct** estimations in cache performance to help you better prepare the examines

Computer Science & Engineering

142

つづく

